



NOTE ... / 20	Nom :	4iir :
---------------	-------	--------------

Instructions :

- ⇒ **Écrivez votre nom, prénom et votre groupe.**
- ⇒ Les réponses doivent être rédigées de manière claire et concise.
- ⇒ Renseigner les réponses dans la feuille de l'examen
- ⇒ Le code fourni doit être corrigé ou complété directement dans les espaces prévus.
- ⇒ Pas de documents autorisés

Exercice 1 : Questions de cours (3pts)

1. Expliquez ce qu'est une interface fonctionnelle en Java et donnez un exemple concret utilisant une expression lambda. **(1pt)**

.....
.....
.....

2. Quelles sont les principales opérations que permet l'API Stream en Java ?
Donnez un exemple pour chacune des opérations suivantes : filtrage, et une méthode réduction de votre choix. **(1pt)**

.....
.....
.....

3. Quelle est la différence entre une exception vérifiée (checked) et une exception non vérifiée (unchecked) ? Donnez un exemple pour chacune. (1 point)

Checked Exception :

.....
Ex :

.....
Unchecked Exception :

.....
Ex :
.....

Exercice 2 : Correction de code (5 pts)

Le code suivant contient des **E**rreurs de **C**ompilation [**EC**] et d'**E**xécution [**EE**]. Repérer l'erreur en indiquant sa nature (**EC/EE**) puis en soulignant le code générant l'erreur et lui donner un numéro, et Corrigez-le.



```
abstract class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public abstract void speak();
}
```

Type de **Erreur**¹ (0.25):

☐ EE | ☐ EC

Code générant l'erreur : (0.25)

.....

Correction proposée : (0.5 pt)

.....

.....

```
class Dog extends Animal {
    public Dog() {}

    public Dog(String name) {
        super(name);
    }

    @Override
    public void speak() {
        System.out.println("Woofwoof");
    }
}
```

Type de **Erreur**² (0.25):

☐ EE | ☐ EC

Code générant l'erreur : (0.25)

.....

Correction proposée : (0.5 pt)

.....

.....

```
class Cat extends Animal
{
    protected String race;

    public Cat(String name, String race) {
        this.race = race; super(name);
    }

    @Override
    public void speak() {
        System.out.println("Meow");
    }

    @Override
    public String toString() {
        return name + " de race " + race;
    }
}
```

Type de **Erreur**³ (0.25):

☐ EE | ☐ EC

Code générant l'erreur : (0.25)

.....

Correction proposée : (0.5 pt)

.....

.....

Type de **Erreur**⁴ (0.25):

☐ EE | ☐ EC

Code générant l'erreur : (0.25)

.....

Correction proposée : (0.5 pt)

.....

.....

.....

.....

.....

.....

.....

.....

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Animal> animals = null;
        animals.add(new Dog("Buddy"));
        animals.add(new Cat("Minou"));

        for (Animal a : animals) a.speak();

        List<Cat> cats = new ArrayList<>();
        cats.add(new Cat("Zoe", "Scottish"));
        cats.add(new Cat("Bella", "Perce"));
        // trie des chats
        Collections.sort(cats);

        for (Cat c : cats) System.out.println(c);
    }
}
```

Type de **Erreur**⁵ (0.25):

☐ EE | ☐ EC

Code générant l'erreur : (0.25)

.....

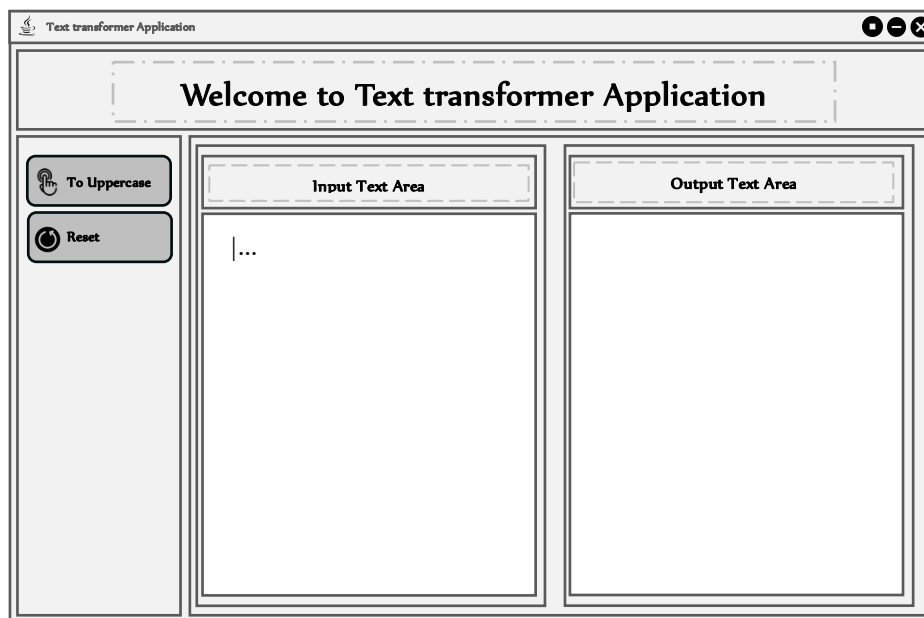
Correction proposée : (0.5 pt)

.....

.....

Exercice 3 : Swing /AWT (4 pts)

1. Soit la fenêtre ci-dessous ; Compléter le tableau en déclarant les différents composants de cette interface graphique swing, (suivre l'exemple de la 1^{ère} entrée du tableau ci-dessous) : (2pts)



Container	Layout Manager	Composants	
JPanel : « pnl_principale »	BorderLayout	JPanel : « pnl_titre » position NORTH JPanel : « pnl_boutons » position WEST JPanel : « pnl_entréeSortie » position CENTER	
JPanel : « pnl_titre »			0.5 pt
JPanel : « pnl_boutons »			0.5 pt
JPanel : « pnl_entréeSortie »			0.5 pt
JPanel : « pnl_entrée »			0.5 pt
JPanel : « pnl_sortie »			

2. Implémenter les actions des boutons (de préférence avec des expressions Lambda (Facultatif)): (2pt)

`btn_toUpperCase.add.....`

`btn_reset.add.....`



Exercice 4 : Lecture et écriture de fichiers (3 pts)

Écrivez un programme Java qui effectue les opérations suivantes :

- ⇒ Lisez un fichier texte "data.txt" ligne par ligne et copiez son contenu dans un autre fichier en ajoutant "Début" comme première ligne du fichier et "Fin" comme dernière ligne du fichier.

Exercice 5 : Stream API - Réflexion et manipulation avancée (5 pts)

Soit la classe Person suivante :

Pour minimiser l'espace, supposant que les getters et setters et le constructeur d'initialisation sont déjà définis

```
class Person {
    String name;
    int age;
    // Constructors
    ... Person(name , age)...
    // Getter et Setter
    ... getAge et getName ...

    public static void main(String[] args) {
        List<Person> persons = new ArrayList<>()
        List.of(new Person("Saad", 22),
                new Person("Ziyad", 23),
                new Person("Sarah", 21),
                new Person("Lilya", 19),
                new Person("Zineb", 24)
                )
    }
}
```

Utilisez l'API Stream pour :

1. Afficher les noms des personnes ayant plus de 22 ans. (1pt)

.....
.....
.....

2. Afficher le nom de la personne la plus âgée. (2pt)

.....
.....
.....

3. Afficher tous les noms triés et en utilisant une méthode référencée (2 pt)

.....
.....
.....